計算物理学II

第3回
LaTeXによる科学文書作成

秋山 進一郎

2026年10月16日

授業日の確認

・全10回

- ・第1回：10月2日（金）
- ・第2回：10月9日（金）
- **・第3回：10月16日（金）**
- ・第4回：10月23日（金）
- ・第5回：11月6日（金）
- ・第6回：11月13日（金）
- ・第7回：11月20日（金）
- ・第8回：12月4日（金）
- ・第9回：12月11日（金）
- ・第10回：12月18日（金）

10月23日は自習とします．各自で演習課題とレポート課題に取り組んでください

今日の授業の目標

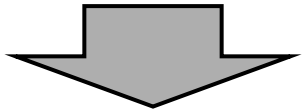
- LaTeXを使う環境を整える
  - シェルスクリプトの利便性を理解する
  - VS codeの設定（授業中）
  - Overleafの利用（授業中 or 授業後）
- 科学文書作成の練習

この授業のレポート課題について

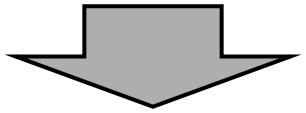
- 4回ともLaTeXで作成して提出してください
  - LaTeXの使用自体を課題の一部とします
- 他の授業のレポート課題でもどんどん使ってみてください

毎回の授業の流れ

“compphy2”に移動する



“git pull origin main”で演習資料を入手



“lecture3”の中の資料を開いて演習開始

本日の演習内容

- LaTeXを使う環境を整える
  - シェルスクリプトの活用
  - VS codeの設定
  - Overleafの利用（時間に余裕があれば）
- LaTeXを使ってみる
  - 数式の記述
  - 図の挿入

LaTeXとは？

- L. Lamportによって開発された文書清書のためのソフトウェア
  - D. Knuthによって開発されたTeX（テック, テフ）を使いやすく拡張したもの
- LaTeXによる基本的な文書作成の流れ

  ① .texファイルを作成, 編集する

  ② .texファイルをplatexコマンドでコンパイルし, .dviファイルを生成する

  ③ dvipdfmxコマンドを使い, .dviファイルを.pdfファイルへ変換する

  ④ 完成した.pdfファイルの確認, 閲覧
- 以下の演習で早速①~④までをやってみます

演習用のディレクトリを準備

- まずは今日の演習用ディレクトリをcompphys2の中に作りましょう
  - pathは「ユーザー名@」以下に書いてありますが, 復習を兼ねてpwdで自分の居場所を確認しておきます

```
akiyama@~/compphys2$ pwd
/Users/akiyama/compphys2
```

  - compphys2の中に“work3”というディレクトリを作ります
    - 作ったらlsで確認しましょう

```
akiyama@~/compphys2$ mkdir work3
akiyama@~/compphys2$ ls
README.md       lecture1        lecture3        python          work1           work3
latex           lecture2        previous_work1  vnc             work2
```

Terminalからコンパイルしてみよう

- 演習で使う.texファイルが“lecture3/src/main.tex”として配布されています
  - “work3”の中に, “project_latex”という作業用のディレクトリを作り, main.texをcpしてきましょう

```
akiyama@~/compphys2/work3$ mkdir project_latex
akiyama@~/compphys2/work3$ cd project_latex
akiyama@~/compphys2/work3/project_latex$ cp ../../lecture3/src/main.tex ./
akiyama@~/compphys2/work3/project_latex$ ls
main.tex
```

① .texファイルを作成, 編集する

- いまcpしてきた“main.tex”を好きなエディタで開きましょう
  - レポート用の雛形としてすぐ使えるようになっています
- まずはこの雛形をそのままコンパイルし, どんな見た目のpdfファイルが出力されるかやってみます
  - エディタを閉じてターミナルへ戻りましょう



② .texファイルをコンパイルし, .dviファイルを生成する

- 全学計算機ではplatexがデフォルトで用意されているはずです
- 以下のようにして, “main.tex”をコンパイルします

```
akiyama@~/compphys2/work3/project_latex$ platex main.tex
```

- ターミナル上に色々と出力されますが, 最後に以下のようなメッセージが出ていれば成功です

```
Output written on main.dvi (1 page, 1172 bytes).
Transcript written on main.log.
```

- lsでディレクトリ内を確認し, “main.dvi”というファイルができていればOK

```
akiyama@~/compphys2/work3/project_latex$ ls
main.aux main.dvi main.log main.out main.tex
```

③ dvipdfmxを使い, .dviを.pdfへ変換する

・次のようにdvipdfmxコマンドを使って，main.dviをmain.pdfに変換します

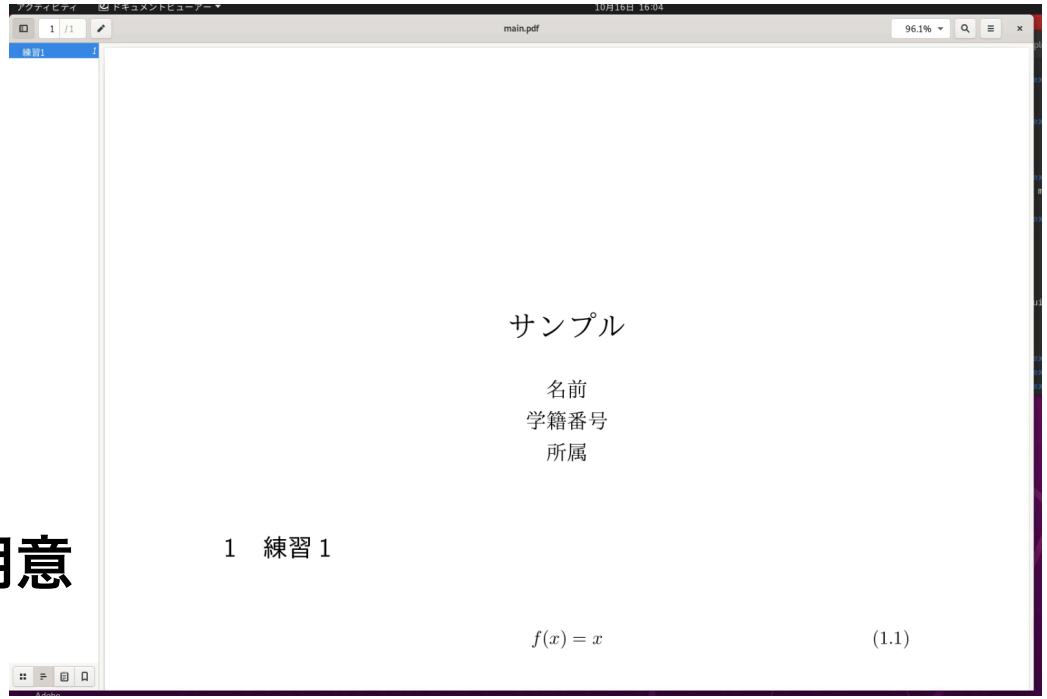
```
akiyama@~/compphys2/work3/project_latex$ dvipdfmx main.dvi
main.dvi -> main.pdf
[1]
11037 bytes written
```

・lsを使い, “main.pdf”というファイルが生成されたことをチェックしましょう

```
akiyama@~/compphys2/work3/project_latex$ ls
main.aux main.dvi main.log main.out main.pdf main.tex
```

④ 完成した.pdfファイルの確認, 閲覧

- 出来上がった“main.pdf”を開いてみましょう
  - 右のようなファイルが見えればOKです
- コンパイルした“main.tex”と見比べてみると, .texファイルのどの部分が各出力に対応しているのか, 何となく見当がつくと思います
  - LaTeXの書き方に関する演習は今日の後半に用意してあります
  - 今後の演習やレポート作成を円滑に進めるために, 先に少し環境設定をやっておきます（次頁へ）

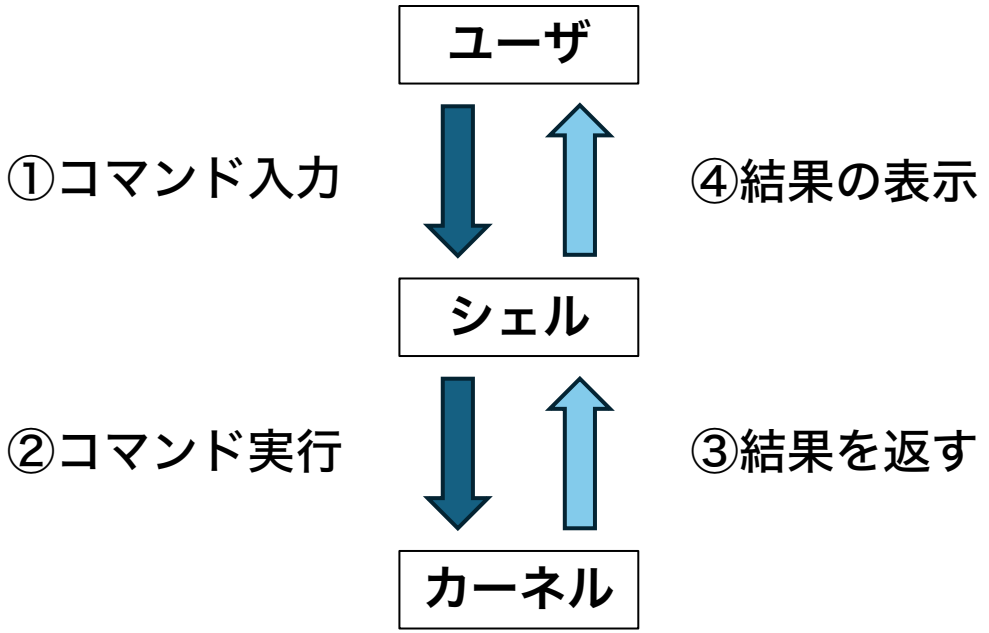


より実践的なLaTeXの使い方

- 今やったように, terminalからtexファルをコンパイルし, pdfファイルを作るには複数のコマンドを立て続けに行う必要がある
  - 実際に文書作成を行う時は頻繁にコンパイルをすることになるので面倒…
  - 文書の作成にはエディタ，コンパイルにはterminal，できた文書の確認にはビューアーを使うため，これらを行き来する必要がある → これも面倒…
- これらの面倒は, 例えば以下のツールを使うことで解消可能です：
  - シェルスクリプト, Perlスクリプト
  - VS code
  - Overleaf
- 以下，今後のLaTeXによる文書作成を快適に行うための準備を進めていきます

スクリプトとは？

- 計算機に実行させたい一連の処理を順序や条件によって記述したもの
  - 具体例として, シェルスクリプト, Perlスクリプト, Pythonスクリプト
- 例えば, シェルスクリプトに記述されたコマンドは, シェルによって解釈・実行され, 必要に応じてカーネルの機能を利用して処理される。（下図参照）
  - シェルスクリプトを予め書いておけば，そこに書かれた複数のコマンドが順番に実行される
  - 同じコマンドを何回も入力しなくて済む
  - Bashの設定により, コマンド名に別名（alias）をつけることも可能
    - “xdg-open”を“open”に変える, など



シェルスクリプトを書いてみよう 1/2

- “work3”の中に“project_shell”という作業用ディレクトリを作り, 再び“main.tex”をcpしてきましょう

```
akiyama@~/compphys2/work3$ mkdir project_shell
akiyama@~/compphys2/work3$ cd project_shell
akiyama@~/compphys2/work3/project_shell$ cp ../../lecture3/src/main.tex .
```

- 先ほどの手順②~④までを一気に行うシェルスクリプトを書いてみます
  - 慣例的に, シェルスクリプトには拡張子.shをつけます
  - シェルスクリプトを書くためのファイルを用意しましょう
    - ここでは“compile_latex.sh”などとしておきました

```
akiyama@~/compphys2/work3/project_shell$ touch compile_latex.sh
```

シェルスクリプトを書いてみよう 2/2

・“compile_latex.sh”をエディタで開き, 以下のようなスクリプトを書き, 保存しましょう

```
#!/bin/bash

platex main.tex
dvipdfmx main.dvi
xdg-open main.pdf
```

・シェルスクリプトを使うには, 実行権限を付与する必要があります

・以下のコマンドによって“compile_latex.sh”に実行権限を与えます

```
akiyama@~/compphys2/work3/project_shell$ chmod +x compile_latex.sh
```

・これでシェルスクリプトの準備が完了しました

シェルスクリプトを実行してみよう

- 実行権限が付与されたスクリプトは, ファイル名の前に“./”をつけて実行できます
  - lsでディレクトリ内を確認した後, この方法でスクリプトを実行してみましょう

```
akiyama@~/compphys2/work3/project_shell$ ls
compile_latex.sh main.tex
akiyama@~/compphys2/work3/project_shell$ ./compile_latex.sh
```

- ビューアーが立ち上がって, コンパイルされた.pdfファイルが見えれば成功です
  - ターミナルに戻って, 再度lsを使い, コンパイルによってファイル生成が行われたことをチェックしましょう

```
akiyama@~/compphys2/work3/project_shell$ ls
compile_latex.sh main.dvi          main.out          main.tex
main.aux         main.log          main.pdf
```

latexmkを使う

- 以下では, LaTeXによる文書作成をより効率化するために, latexmkコマンドを使います
  - latexmkでは, LaTeXのコンパイルに必要な処理が自動的に判断され, 必要に応じて適宜コンパイルを繰り返してくれます
  - 例えば, LaTeXで参考文献を載せたりする場合には複数回のコンパイルが必要になるのですが, latexmkを使えばこのコマンド一つで片付けることができるので重宝します
- latexmkの動作は, “.latexmkrc”というPerlスクリプトによって設定します
  - Perlスクリプトとは, Perlというプログラミング言語によって記述されたスクリプトのことです

Perlスクリプトを準備する 1/2

- Perlスクリプトは“lecture3/src/.latexmkrc”として準備されています
  - まずは“lecture3/src”へ移動しましょう
  - 名前が“.”から始まるファイルは隠しファイルと呼ばれ, 単純なlsからは見えません

```
akiyama@~/compphys2/lecture3/src$ ls
main.tex                settings.json          template_overleaf.zip
```

  - lsに-aオプションをつけると見えるようになります

```
akiyama@~/compphys2/lecture3/src$ ls -a
.                       .latexmkrc             settings.json
..                      main.tex               template_overleaf.zip
```

Perlスクリプトを準備する 2/2

- .latexmkrcを全学計算機のホームディレクトリへcpします

```
akiyama@~/compphys2/lecture3/src$ cp .latexmkrc ~/
```

- 以上でスクリプトの準備はOKです
- それではスクリプトを使ったコンパイルをやってみます
  - “work3”へ戻って, スクリプトによるコンパイル用の作業ディレクトリを作り, 先ほど使ったソースコード“main.tex”をcpしてきます

```
akiyama@~/compphys2/work3$ mkdir project_latexmk
akiyama@~/compphys2/work3$ cd project_latexmk
akiyama@~/compphys2/work3/project_latexmk$ cp ../../lecture3/src/main.tex .
akiyama@~/compphys2/work3/project_latexmk$ ls
main.tex
```

.latexmkrcの詳細について 1/3

- “.latexmkrc”をcatで一度見ておきましょう
- この授業ではPerlの詳細には立ち入らないので, スクリプトの内容を正確に理解する必要はありません
- $latex：LaTeX のコンパイルに使用するコマンドを指定
- platex：pLaTeX を使って文書をコンパイル
- -synctex=1：ソースコードと .pdfファイル上の位置の対応付けを有効にする
- -halt-on-error：エラーが発生したらコンパイルを停止
- -file-line-error：エラーをファイル名と行番号付きで表示
- %O：latexmk が指定する追加オプションに置き変わる
- %S：コンパイル対象のソースファイル名に置き換わる
- $max_repeat = 5：必要に応じてコンパイルを上限5回で繰り返す

```
akiyama@~/compphys2/lecture3/src$ cat .latexmkrc
#!/usr/bin/env perl

$latex = 'platex -synctex=1 -halt-on-error -file-line-error %O %S';
$max_repeat = 5;
```

.latexmkrcの詳細について 2/3

- 参考文献情報の処理：
```
$bibtex = 'pbibtex %O %S';
$biber = 'biber --bblencoding=utf8 -u -U --output_safechars %O %S';
```
- 索引を作成：
```
$makeindex = 'mendex %O -o %D %S';
```
- .dviから.pdfへの変換：
```
$dvipdf = 'dvipdfmx %O -o %D %S';
$pdf_mode = 3;
```
- 出力先ディレクトリの指定：
```
$out_dir = 'output';
```
  - “output”という名前のディレクトリを作成し, コンパイル時に生成されるファイル群を全て格納する
- ビューアーに関する設定：
```
$pvc_view_file_via_temporary = 0;

if ($^O eq 'linux') {
    $dvi_previewer = 'xdg-open %S';
    $pdf_previewer = 'xdg-open %S';
} elsif ($^O eq 'darwin') {
    $dvi_previewer = 'open %S';
    $pdf_previewer = 'open %S';
} else {
    $dvi_previewer = 'start %S';
    $pdf_previewer = 'start %S';
}
```

.latexmkrcの詳細について 3/3

- クリーンアップに関する設定： `$clean_full_ext = '%R.synctex.gz';`
  - “latexmk –c”で“output”内に格納されている補助ファイル群を削除
  - “latexmk –C”で“output”内に格納されている生成されたファイルを全て削除

latexmkで一発でコンパイルする

- latexmkコマンドを実行することで，先ほどホームに置いた.latexmkrcが実行されます

```
akiyama@~/compphys2/work3/project_latexmk$ latexmk main.tex
```

- 先ほど同様ターミナルに色々と出力がありますが, 最後に次のようなメッセージが出力されていれば成功です

```
output/main.dvi -> output/main.pdf
[1]
11092 bytes written
Latexmk: All targets (output/main.dvi output/main.pdf) are up-to-date
```

- lsで確認し, “output”というディレクトリが出来たことを確認しましょう
  - “output”の中へ移動し, “main.pdf”やその他の補助ファイルが作られていることを確認しましょう

```
akiyama@~/compphys2/work3/project_latexmk$ ls
main.tex output
```

クリーンアップしてみましょう

- latexmkを実行した場所では, 以下のコマンドによってクリーンアップできます
  - “latexmk –c”：“output”内に格納されている補助ファイル群を削除
  - “latexmk –C”：“output”内に格納されている生成されたファイルを全て削除
- “latexmk –C”は生成物を全てリセットして, 最初からコンパイルし直すのに便利です
  - コンパイルエラーが生じた後に使うと良いです
- “latexmk –C”をして, “output”が空になったことを確認し, 再度コンパイルをやってみましょう

```
akiyama@~/compphys2/work3/project_latexmk$ latexmk -C
Rc files read:
  /Users/akiyama/.latexmkrc
Latexmk: This is Latexmk, John Collins, 31 Jan. 2024. Version 4.83.
Latexmk: Doing main (small) clean up for 'main.tex'

akiyama@~/compphys2/work3/project_latexmk$ ls
main.tex output
akiyama@~/compphys2/work3/project_latexmk$ cd output
akiyama@~/compphys2/work3/project_latexmk/output$ ls
akiyama@~/compphys2/work3/project_latexmk/output$
```

VS codeの利活用

- スクリプトによりコンパイルの手順②と③が統合されました
- ただし, この状態では, まだ.texファイルを編集する度に出来上がった.pdfファイルをターミナルから開かないといけません
- この授業ではVS codeを使うことで, .texファイルの編集と.pdfファイルの閲覧を同時に行えるようにしたいと思います
  - VS codeでは便利な拡張機能が色々使えます
  - Agent coding（AIエージェントによるプログラム開発）とも相性が良い
- いったん“compphys2”のディレクトリまで戻り, 以下のコマンドを実行してVS codeを立ち上げましょう

```
akiyama@~/compphys2$ code
```

VS codeの設定 1/5

・以下の手順で，拡張機能であるLaTeX Workshopをインストールする

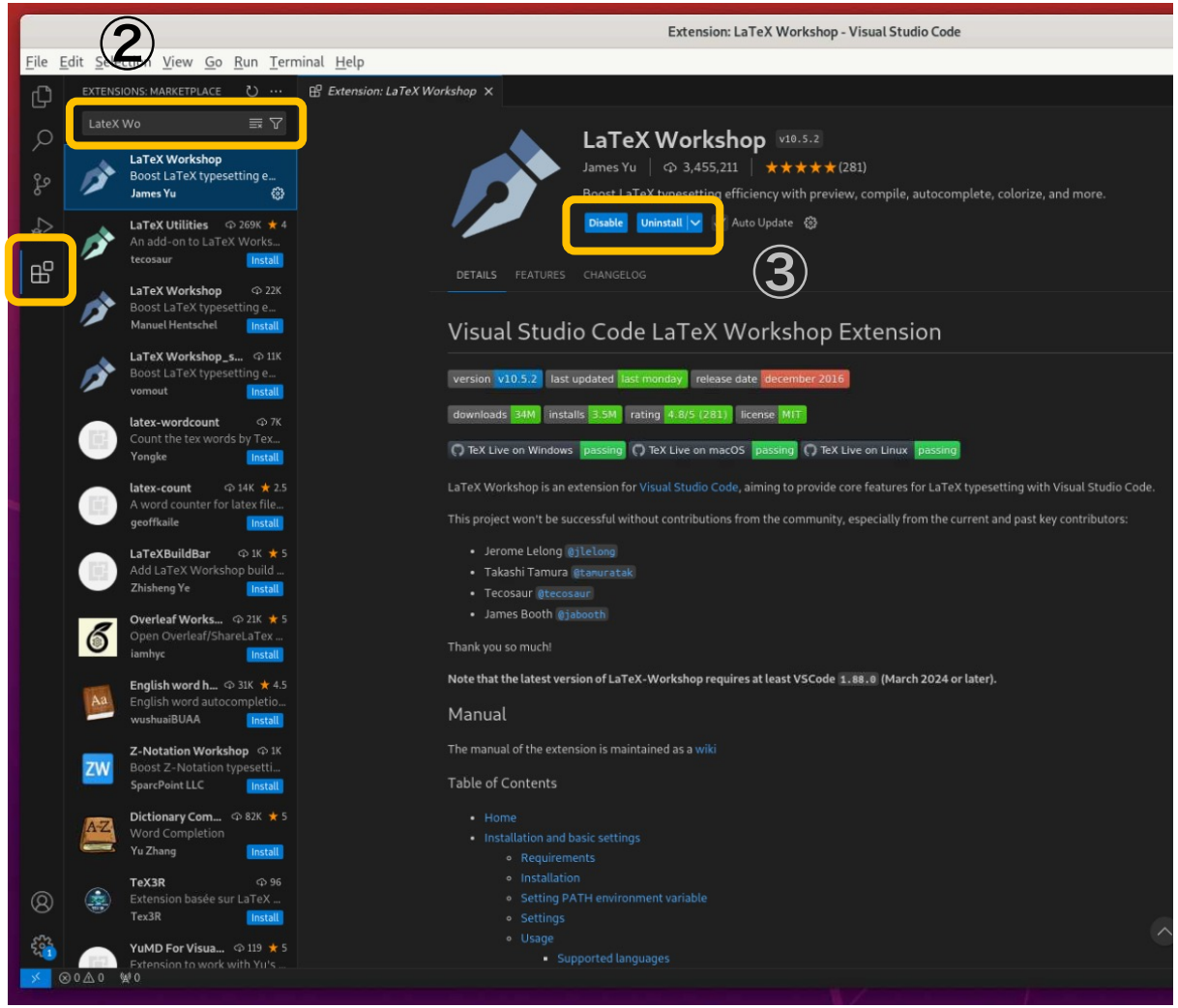
① Extensionsをクリック

② 検索窓に「latex」と入力する

- 検索結果の中から
「LaTeX Workshop」を選択します



③ 青いinstallボタンをクリック
（右図ではinstall済となっている）



VS codeの設定 2/5

- ここで一度terminalに戻ります
  - VS codeの設定は, settings.jsonというファイルを通して行います
  - setting.jsonはVS codeのアプリからも開くことができますが, ここでは練習も兼ねてterminalから行います
- すでに編集されたsettings.jsonがlecture3/srcの中に入っています

```
akiyama@~/compphys2/lecture3/src$ ls
main.tex                settings.json          template_overleaf.zip
```

VS codeの設定 3/5

- ホームディレクトリへ移動し, .config/Code/Userというディレクトリへ入ります
  - lsで中を見てみましょう
  - 人によっては, すでにsettings.jsonがあるかもしれませんが, 問題ありません

```
akiyama@~/compphys2/lecture3/src$ cd
akiyama@~$ cd .config/Code/User/
akiyama@~/.config/Code/User$ ls
chatLanguageModels.json  globalStorage  History  settings.json  snippets  workspaceStorage
```

- 配布されているsettings.jsonをここへcpしてきます

```
akiyama@~/.config/Code/User$ cp ~/compphys2/lecture3/src/settings.json .
```

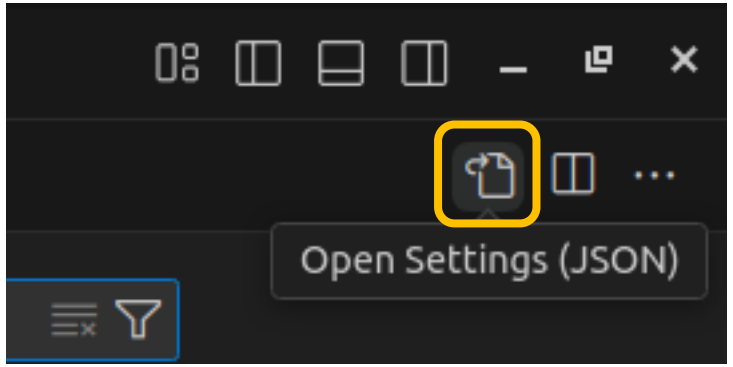
- ここまでできたらVS codeの画面に戻ります

VS codeの設定 4/5

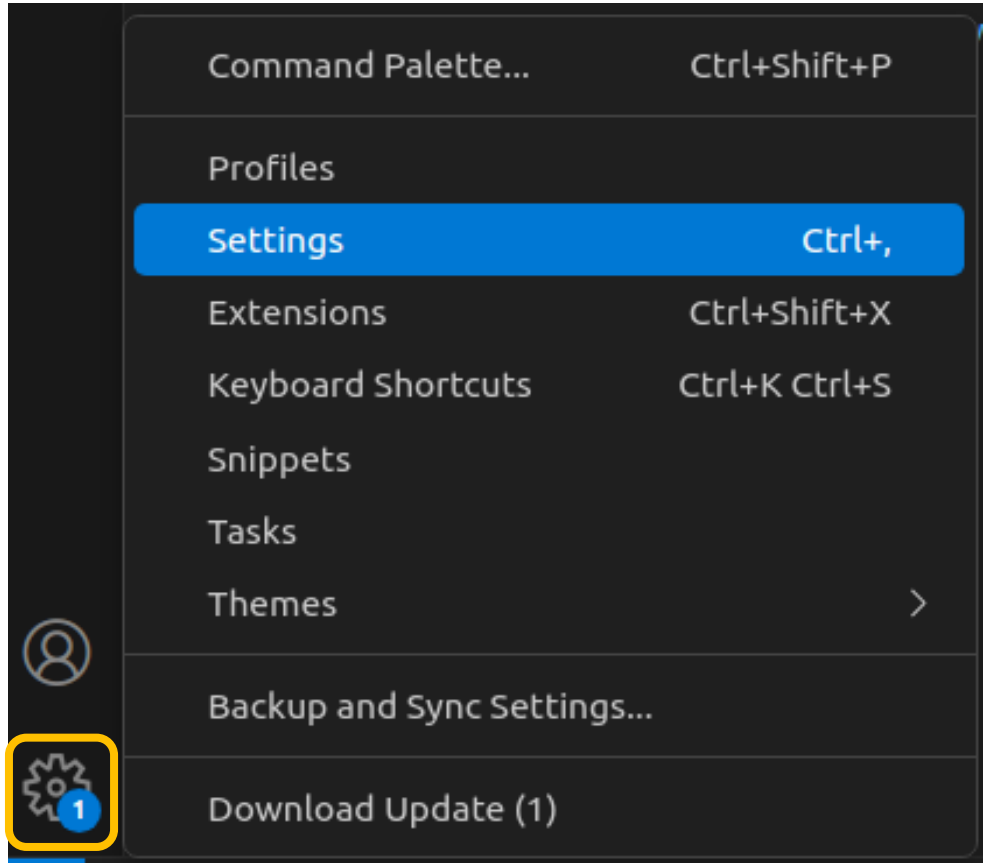
・先ほどcpしたsettings.jsonをVS codeから確認します

① VS codeの左下にある  マークを押し，“Settings”を選択（右図参照）

② VS codeの右上に“Open Settings(JSON)”が表示されるので選択（下図参照）



②



①

VS codeの設定 5/5

- 左のようなjsonファイルが見えればOKです
  - 内容の詳細な理解は今回は不要です
  - 「.jsonファイルを通して, LaTeXの環境を設定できる」ということを頭に入れておいてもらえればOKです
- ここまでできたら, 一度VS codeを終了します
  - これでVS codeの設定が完了しました



VS codeを使ってみよう 1/3

- それではVS codeを使ってLaTeXの編集, コンパイルをしてみましょう
  - “work3”に戻り, “project_vscode”というディレクトリを用意します
  - “project_vscode”の中に“main.tex”をcpしてきましょう

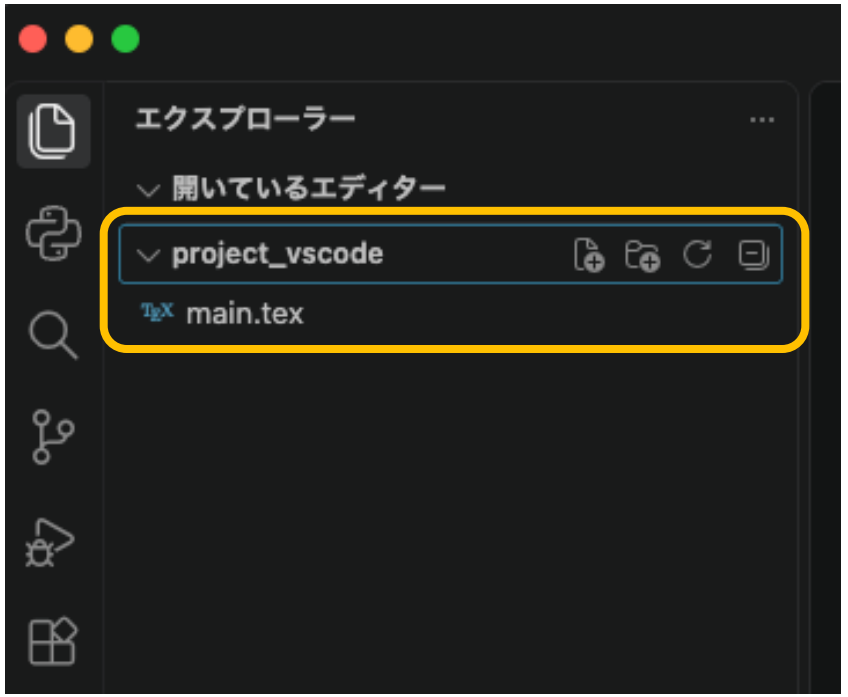
```
akiyama@~/compphys2/work3$ mkdir project_vscode
akiyama@~/compphys2/work3$ cd project_vscode
akiyama@~/compphys2/work3/project_vscode$ cp ../../lecture3/src/main.tex .
akiyama@~/compphys2/work3/project_vscode$ ls
main.tex
```

  - “project_vscode”のディレクトリごとVS codeで開きます
    - codeコマンドの後ろに開きたいディレクトリを入力して実行します

```
akiyama@~/compphys2/work3/project_vscode$ code .
```

VS codeを使ってみよう 2/3

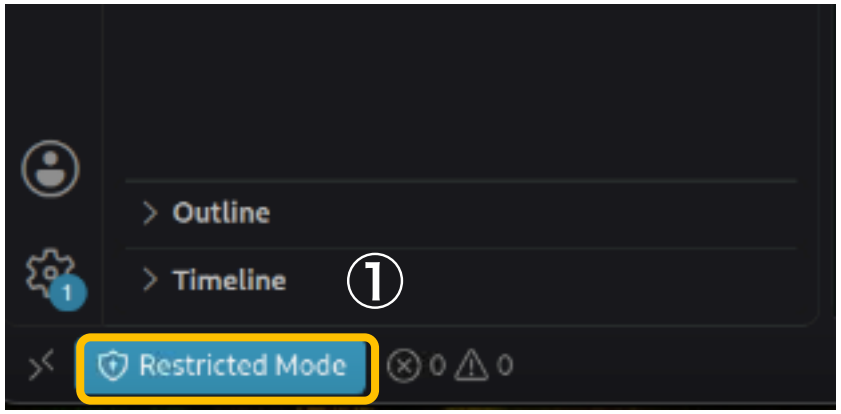
- 左のエクスプローラーからディレクトリを見ることができます
  - “main.tex”をクリックすると, 編集画面で開きます
- texファイルを開いた状態で，Ctrl+Alt+Bを押すとコンパイルされます ⇒ やってみましょう
- Ctrl+Alt+Vを押すと右にプレビューが表示されます ⇒ やってみましょう
- これらのコマンドの代わりに, 右上のボタン（下図）を使うこともできます



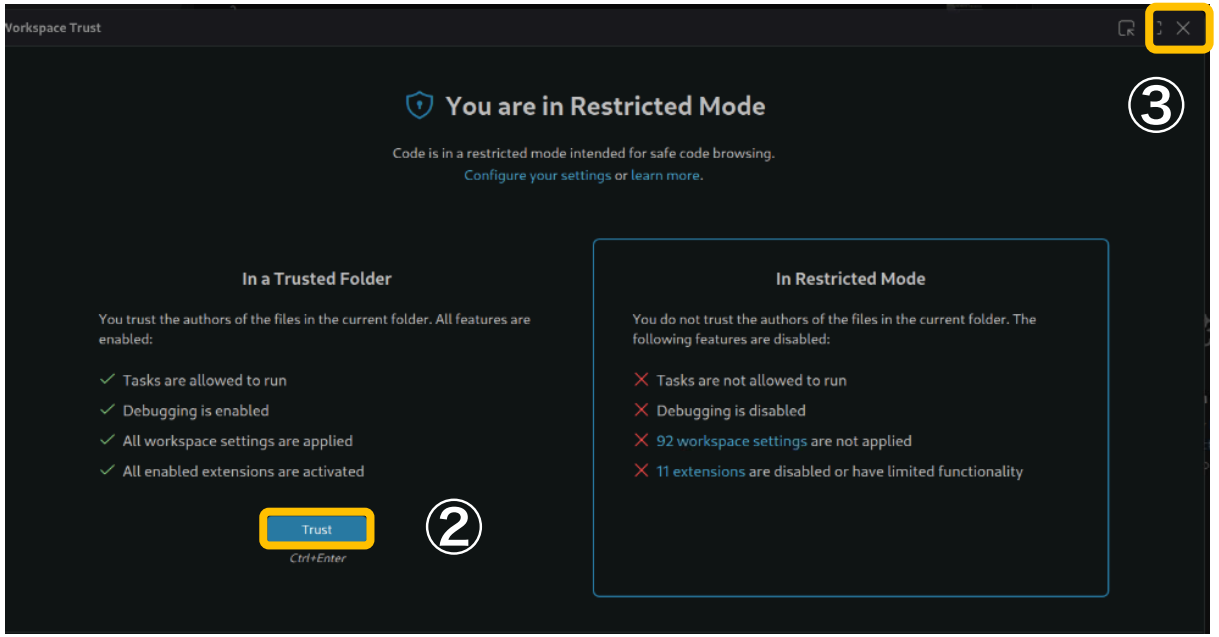


コンパイルができない場合

- p.33の最後で, ディレクトリごとVS codeで開く際に, このディレクトリが「信頼できない」と自動的に判断される場合があります
  - VS codeの画面左下で“Restricted Mode”となっていると, 今開いているディレクトリが信頼されていません
  - このままだとコンパイルできません！

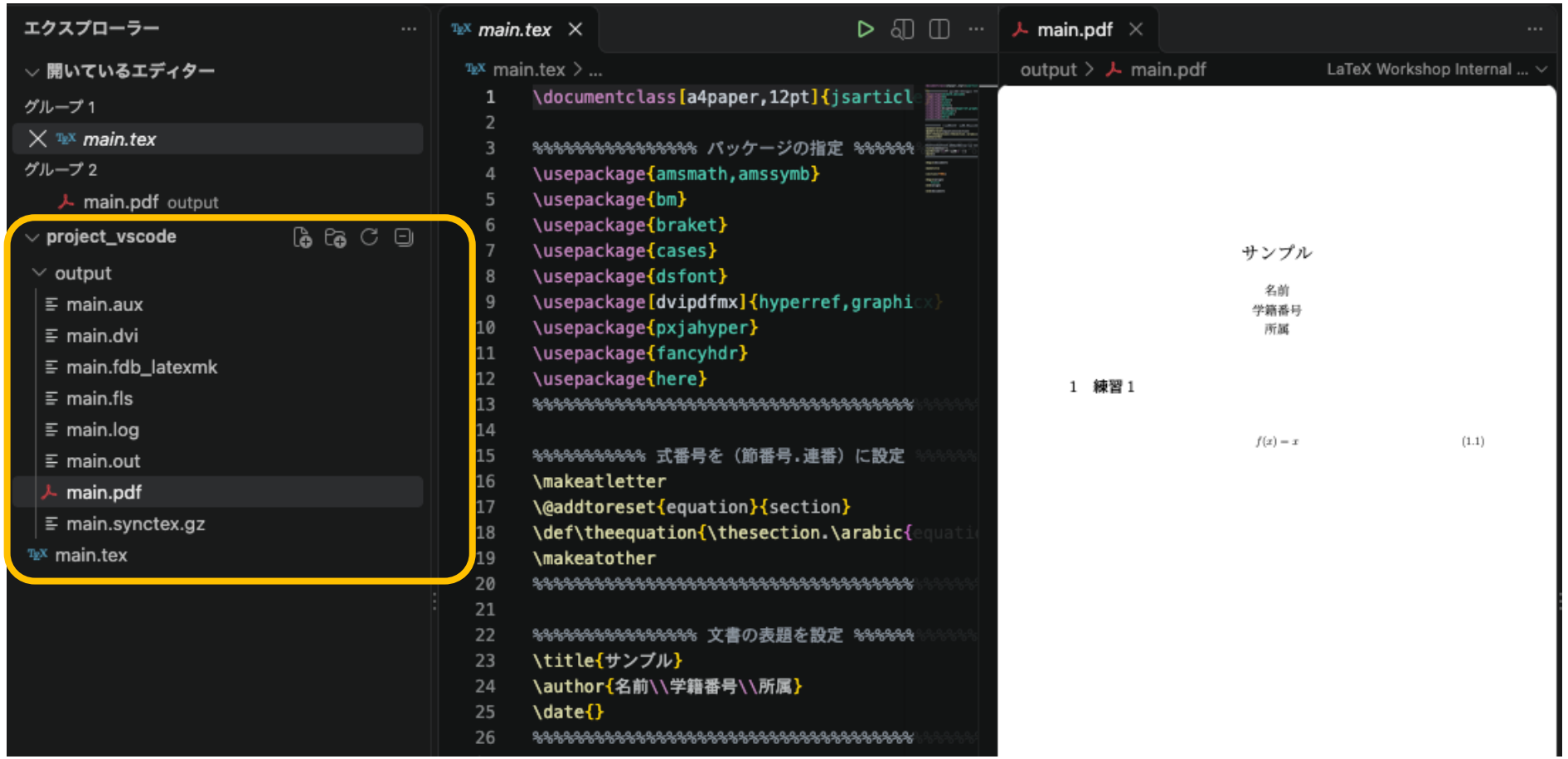


- 以下の手順で“Restricted Mode”を解除します
  - ① “Restricted Mode”をクリック
  - ② 左側の“Trust”をクリック
  - ③ 右上の“X”ボタンからこのウィンドウを閉じる
  - これでコンパイルできるようになるはず



VS codeを使ってみよう 3/3

- コンパイルが通ると, outputというディレクトリが作られ, その中に“main.pdf”をはじめ, 補助ファイル群が格納されます



VS codeからターミナルを開くこともできます

- 左上の「Terminal」→「New Terminal」を押すかCtrl+Shift+Oを押すとVS codeからターミナルを使うこともできます



- これでターミナルとVS codeを行き来する手間もなくなります
  - 例えば, VS codeのターミナルからクリーンアップをしてみましょう



これでVS codeの準備は完了

- 以上の準備は一回だけやればよく, 今後はVS codeを使って.texファイルの編集だけに注力することができます

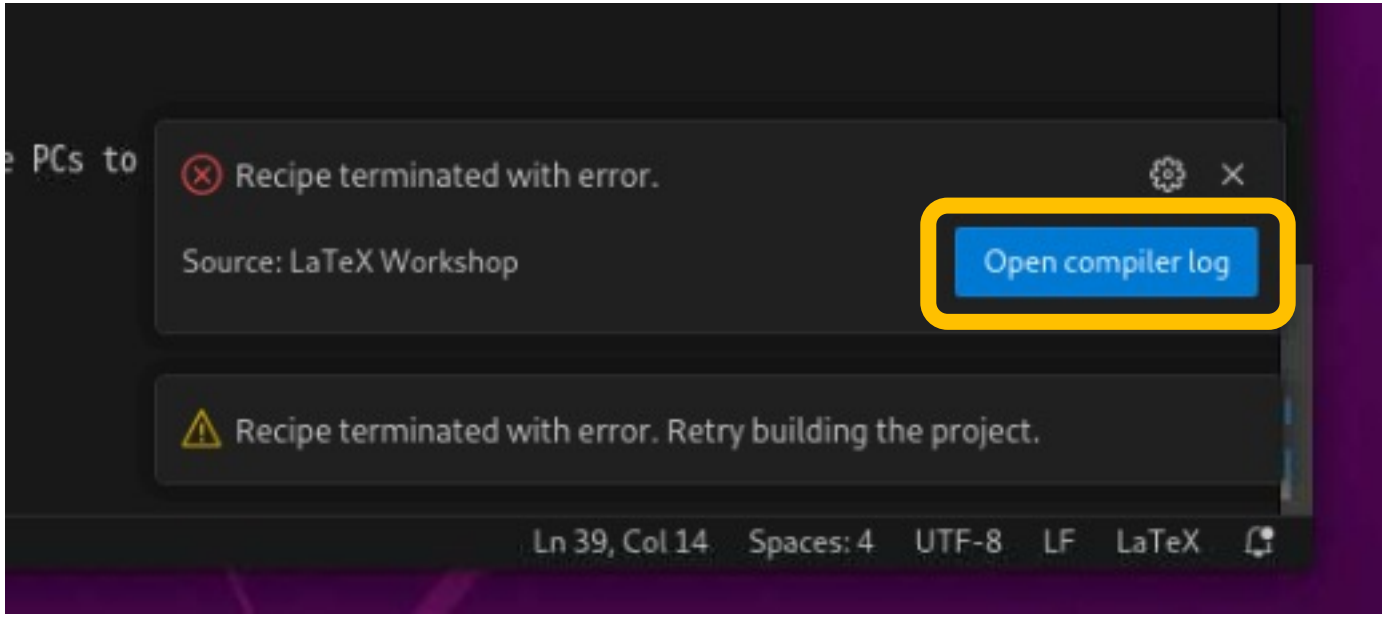
- それでは, VS codeを使ってLaTeXによる文書作成を実際に進めていきましょう

本日の演習内容

- LaTeXを使う環境を整える
  - シェルスクリプトの活用
  - VS codeの設定
  - Overleafの利用（時間に余裕があれば）
- **LaTeXを使ってみる**
  - **数式の記述**
  - **図の挿入**

以降の演習について

- 今開いているVS codeを使っていきます
- texファイルを編集したら，こまめにCtrl+Alt+Bでコンパイルしましょう
- コンパイルエラーになると，それを知らせるウィンドウが右下に現れます
  - 「Open compiler log」を押して，メッセージを見ましょう
  - エラー箇所の特定につながります

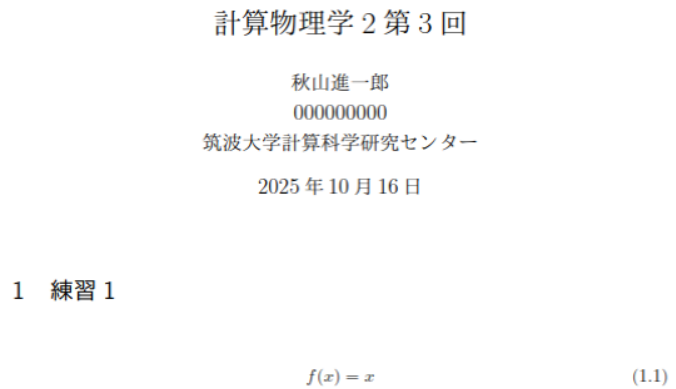


文書の表題を設定しよう

- タイトル，名前，学籍番号，所属，日付を入力し，main.texを上書き保存しましょう
  - ここでは仮に「計算物理学2第3回」としました
  - 日付はハードコーディングせずに，＼todayを使いましょう
  - もし日本語入力する方法が分からなければ，ここ を参照

保存せずにコンパイルすると
編集内容が反映されないので注意

```
22  %%%%%%%%%%%%%%%% 文書の表題を設定 %%%%%%%%%%%%%%%%
23  \title{計算物理学2第3回}
24  \author{
25      秋山進一郎\\
26      000000000\\
27      筑波大学計算科学研究センター
28  }
29  \date{\today}
30  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```



計算物理学 2 第 3 回

秋山進一郎
000000000
筑波大学計算科学研究センター

2025 年 10 月 16 日

1　練習 1

$$f(x) = x \tag{1.1}$$

数式を入力しよう

- 例として，f(x)=xという式が入力してあります
- 配布資料「LaTeX入門」の3節, 7節, 8節を参照し，以下の課題に取り組みましょう
  - 式(1.1)の下にf(x)=sinxを出力させましょう
  - さらにその下にsinxの原点周りでのTaylor展開を書いてみましょう

1　練習 1

$$f(x) = x \tag{1.1}$$

$$f(x) = \sin x \tag{1.2}$$

$$\sin x = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \tag{1.3}$$

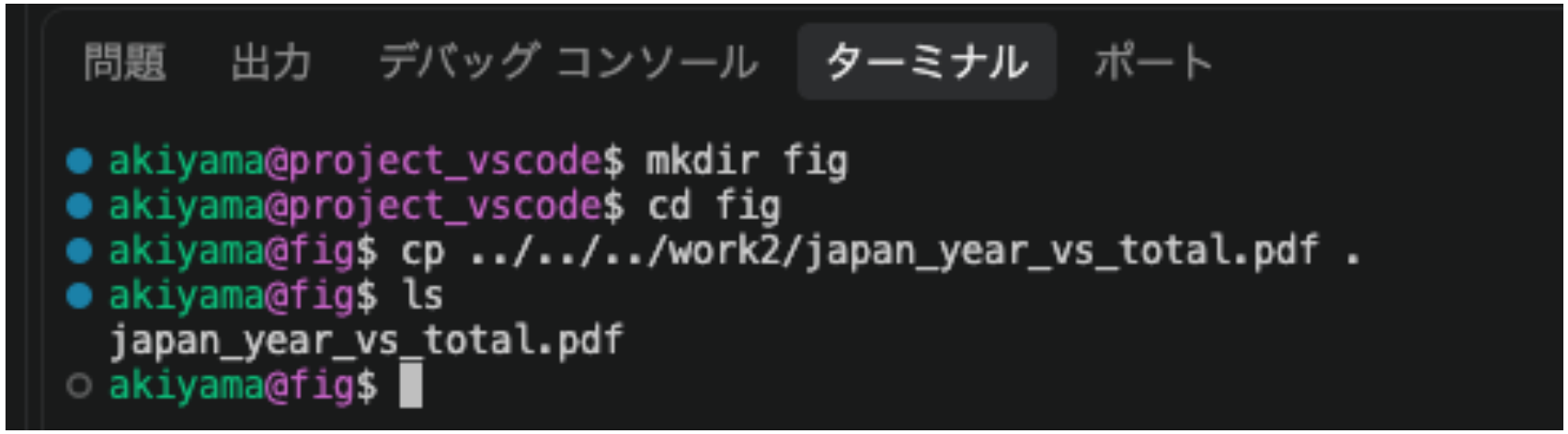
図を挿入してみよう 1/2

・第2回の演習で作成した図を挿入してみましょう

・今いる“project_vscode”の中に“fig”という名前のディレクトリを作りましょう

　・VS codeで開いているTerminalを使ってみましょう

・第2回の演習で作成した図を“fig”にcpしてきましょう



図を挿入してみよう 2/2

- .texファイル上に新しいsectionを作りましょう
  - ＼section{練習2}などとしましょう
- 「LaTeX入門」の6節を見ながら，先ほどcpしてきた図を挿入しましょう
  - ＼caption{}に図の説明を記述してみましょう
  - コンパイルして図が表示されましたか？

特殊文字を含む数式を書いてみよう

- .texファイル上に新しいsectionを作りましょう
  - ＼section{練習3}などとしましょう
- 「LaTeX入門」を適宜参照しながら，下のような文章とともにSchrödinger方程式を書いてみましょう（öは＼"{o}）

3　練習 3

量子力学で基本となる方程式は，Schrödinger 方程式と呼ばれ，

$$\mathrm{i}\hbar\frac{\partial\Psi}{\partial t} = -\frac{\hbar^2}{2m}\nabla^2\Psi + V\Psi \tag{3.1}$$

で与えられる．ここで，$\hbar$ は

$$\hbar = \frac{h}{2\pi} = 1.054572 \times 10^{-34}\mathrm{Js} \tag{3.2}$$

である．

ここまで終わったら

・お疲れ様でした

・時間に余裕のある人は, 以降のページに従い，Overleafも使ってみましょう

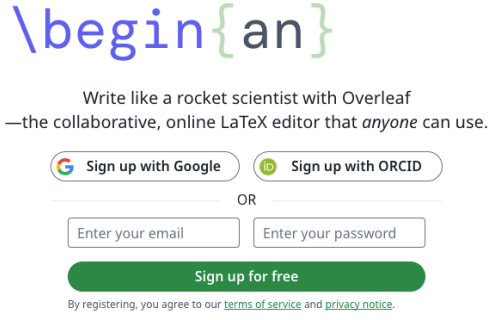
Overleafとは？

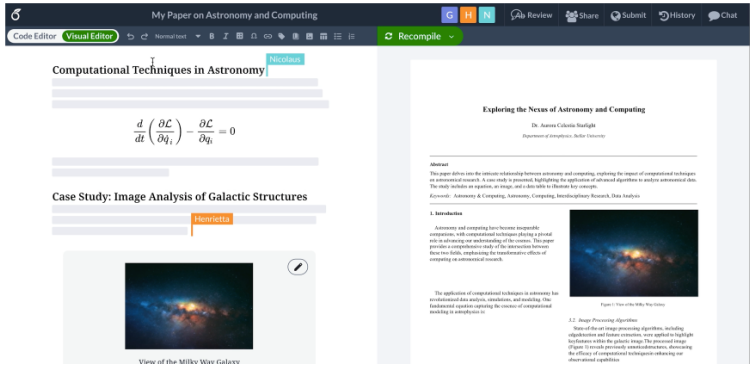
- LaTeX専用のオンラインエディタ
  - 環境構築が不要
  - クラウドサービスなので，ネット環境さえあればどこからでも作業可能
  - クラウドサービスなので，自分のパソコンが壊れてもデータは残る
  - GitHubとの連携ができる（生成AIと組み合わせて使う場合に便利）
  - 共同編集者を追加できる（無料プランでは1人まで）
  - 無料プランの場合，図の量が増え過ぎるとコンパイルが遅くなるので注意
- 授業時間外では，リモートデスクトップでVS codeを使うよりも，素直にブラウザからOverleafを使う方がラクかもしれません

Overleafのアカウントを作る

- Overleafにアクセスし，sign upをクリック
  - メイルアドレス，パスワードの登録をする
  - 初めて登録する人は無料プランを選び，登録手続きを進めましょう

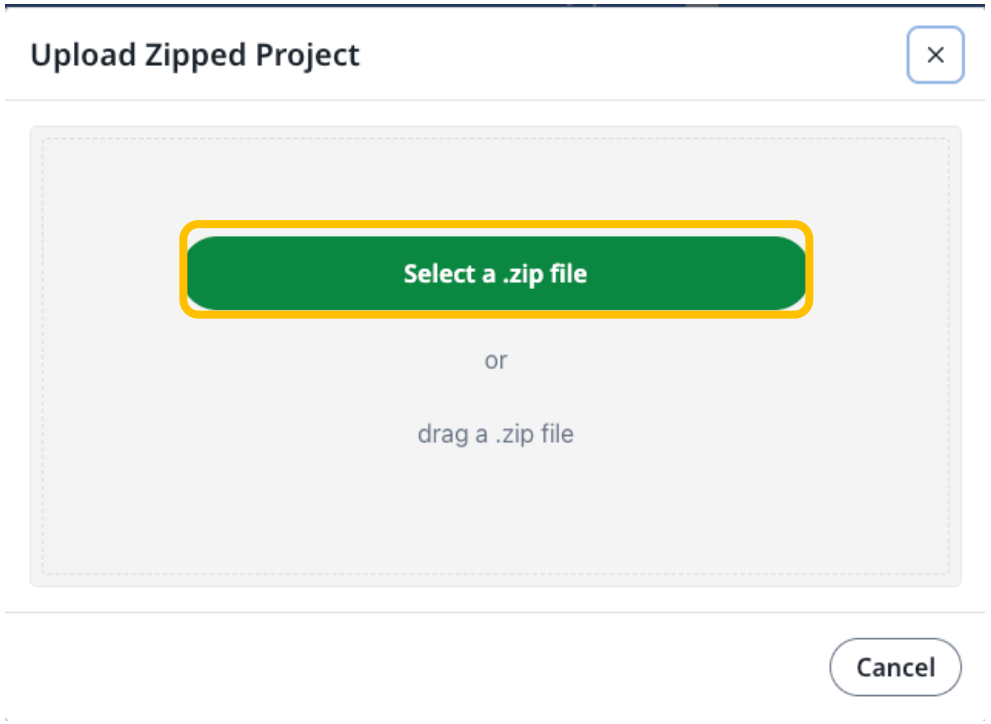


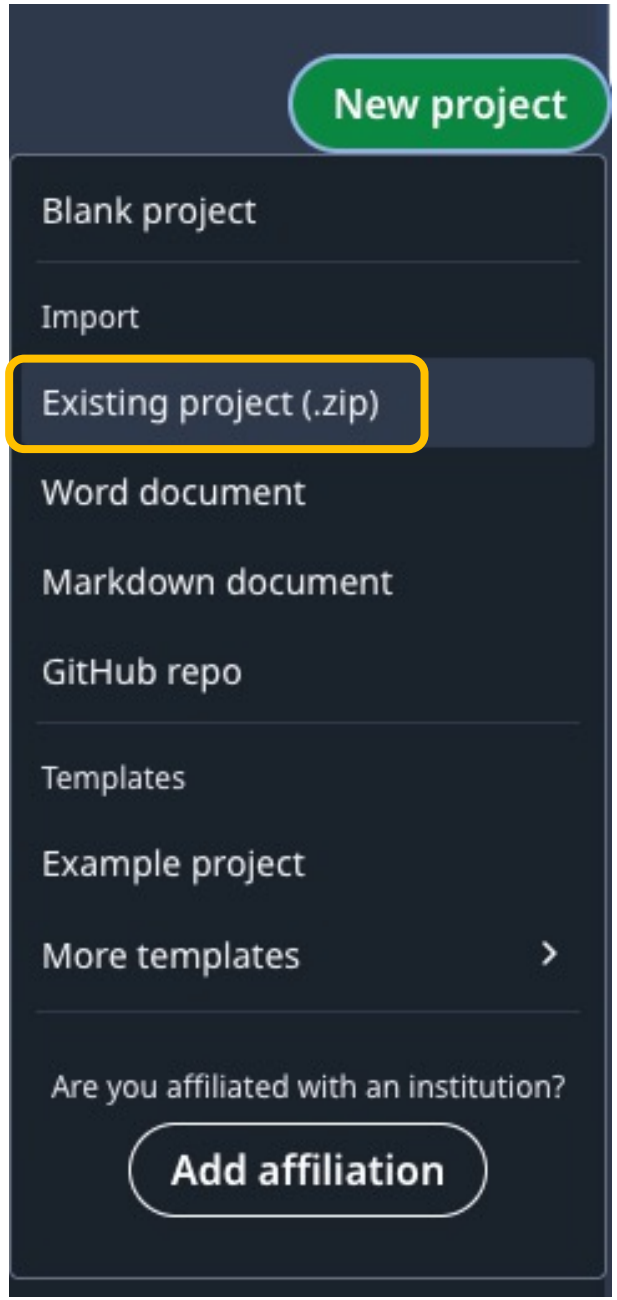




OverleafでNew Projectを作る

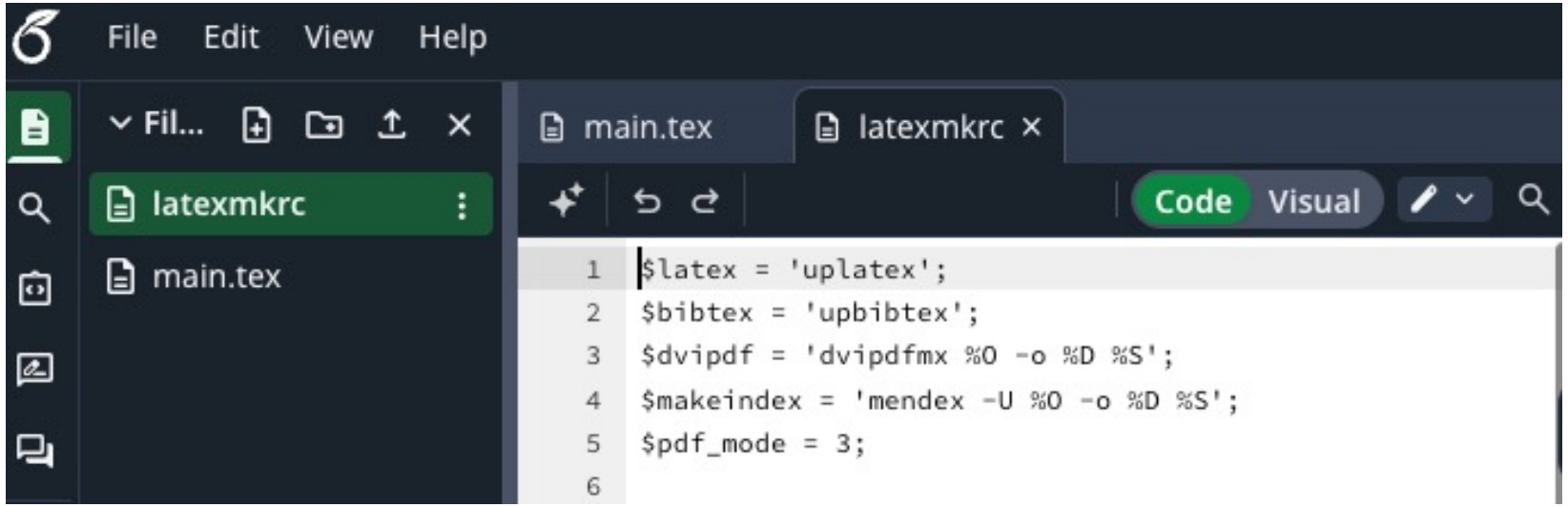
- OverleafにLog inしたらNew Projectを作ってみよう
  - 作り方は色々あるが，ここではExisting projectを選ぶ
  - 以下のように, lecture3/srcにある“template_overleaf.zip”をアップロード





Overleafで日本語を使えるようにする 1/3

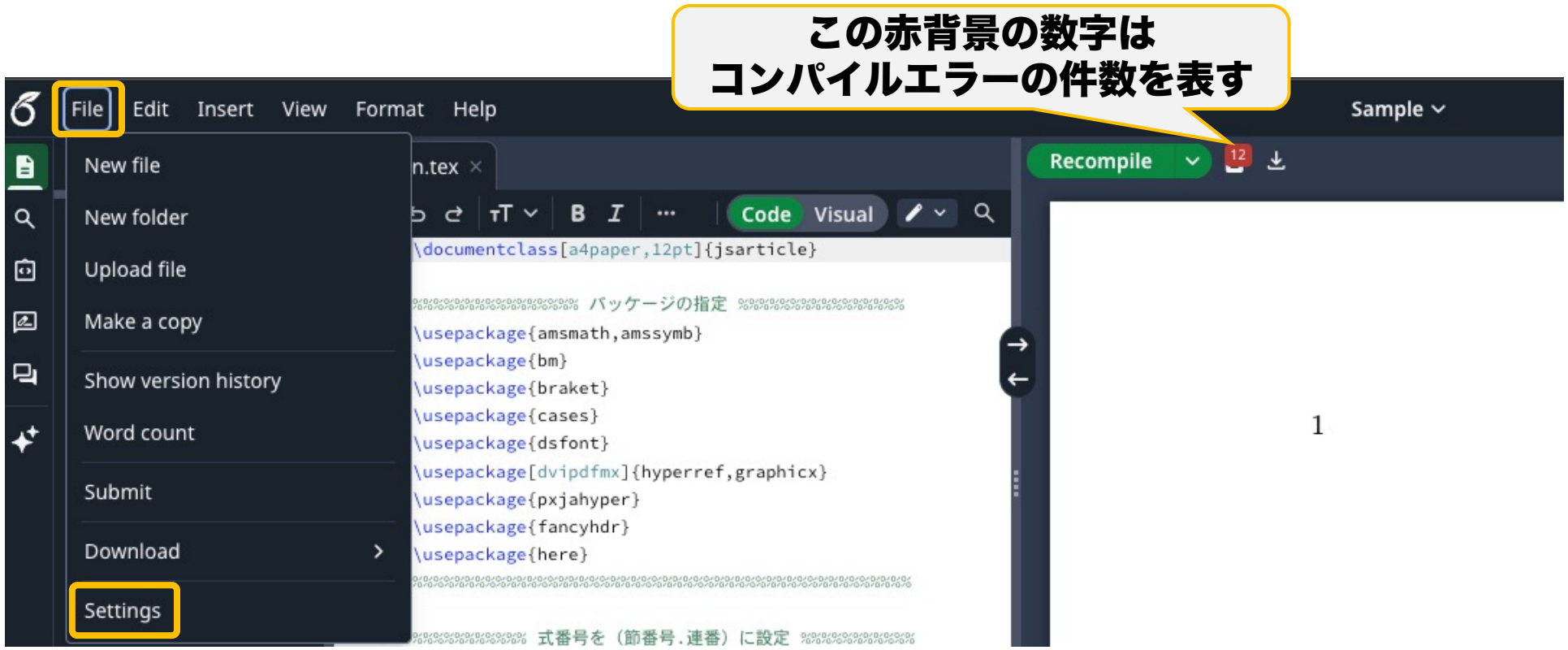
- 日本語を使うためにはlatexmkrcとcompilerの変更が必要
  - まず, Projectのディレクトリ内にlatexmkrcを用意する
  - 今回は, こちらで準備したlatexmkrcをtemplate_overleaf.zipに入れておいたので準備不要



Overleafで日本語を使えるようにする 2/3

- Overleafはデフォルトでは日本語の入出力ができず, エラーが出る
- 以下のようにして設定を変えます

① 左上の「File」を押し, ②「Settings」選択

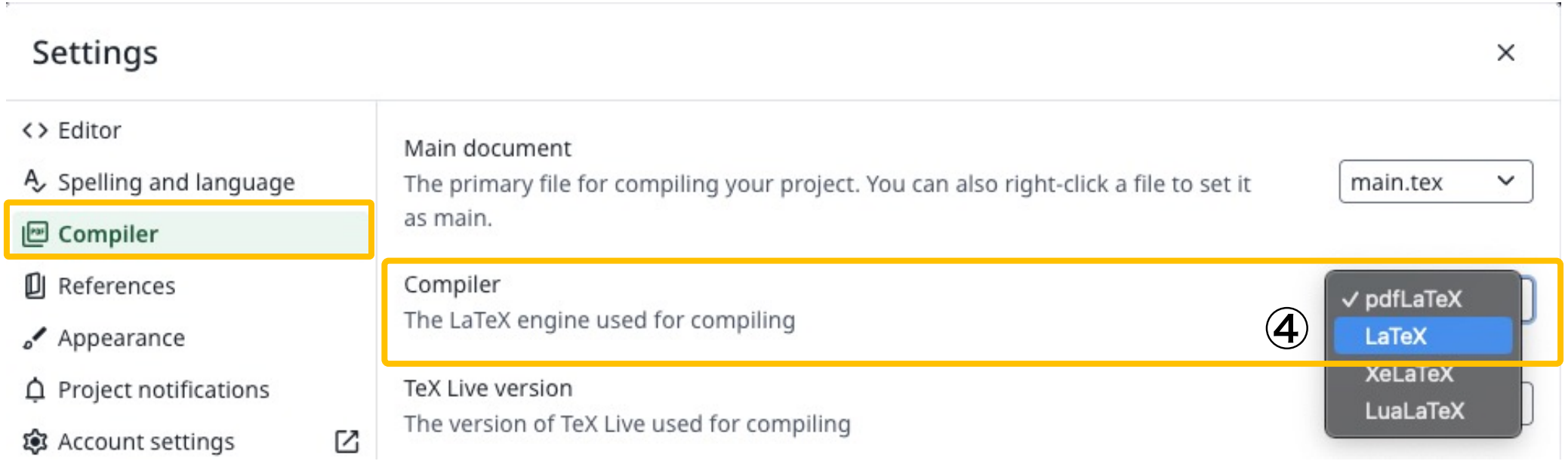


Overleafで日本語を使えるようにする 3/3

・以下のようにして設定を変えます（続き）

③「Compiler」を選択

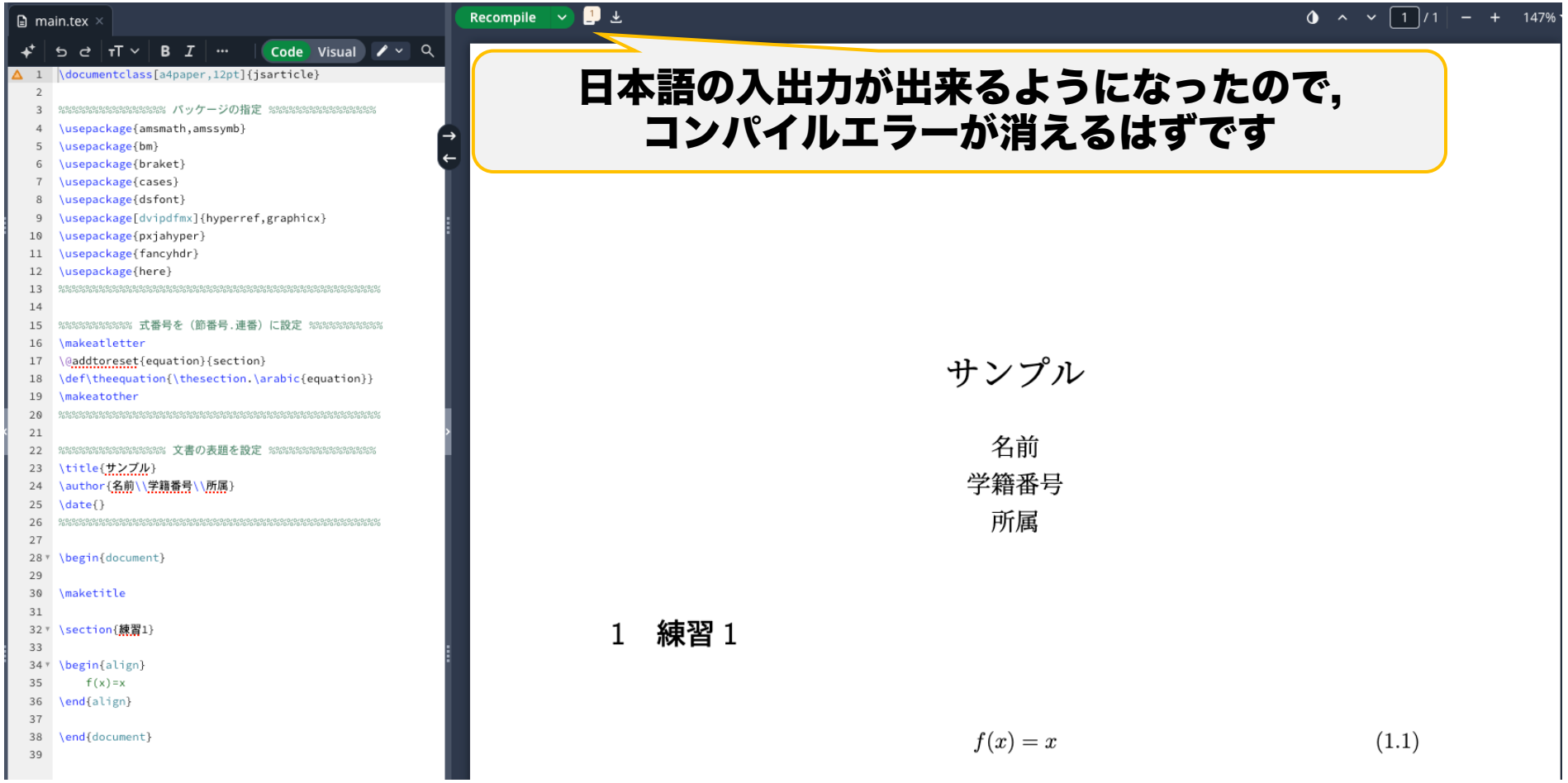
④ Compilerをデフォルトの「pdfLaTeX」から「LaTeX」へ変更



・以上で準備完了です

Overleafでのコンパイル

・Ctrl+Sでコンパイルと保存が実行されます. やってみましょう




```
\documentclass[a4paper,12pt]{jsarticle}

%%%%%%%%%%%%%%%%%%%% パッケージの指定 %%%%%%%%%%%%%%%%%%%%
\usepackage{amsmath,amssymb}
\usepackage{bm}
\usepackage{braket}
\usepackage{cases}
\usepackage{dsfont}
\usepackage[dvipdfmx]{hyperref,graphicx}
\usepackage{pxjahyper}
\usepackage{fancyhdr}
\usepackage{here}
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%% 式番号を（節番号.連番）に設定 %%%%%%%%%%%%%%%%
\makeatletter
\@addtoreset{equation}{section}
\def\theequation{\thesection.\arabic{equation}}
\makeatother
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%% 文書の表題を設定 %%%%%%%%%%%%%%%%%%%%
\title{サンプル}
\author{名前\\学籍番号\\所属}
\date{}
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

\begin{document}

\maketitle

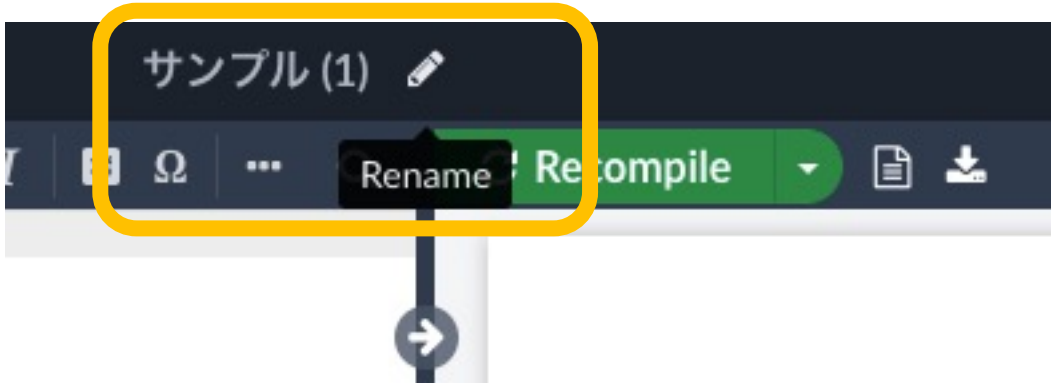
\section{練習1}

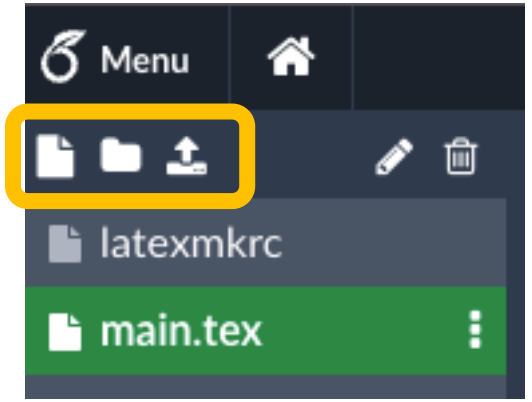
\begin{align}
    f(x)=x
\end{align}

\end{document}
```

Overleafの使い方

- Ctrl+Sでコンパイルと保存が実行される
- Project名の変更は画面中央上から
- 「Menu」の下のボタンから新規ディレクトリ，ファイルの作成，ファイルのアップロードが可能
- 保存していない状態でブラウザを離れようとすると保存するか聞いてくるのでよく確認すること





参考サイト

・TeX Wiki　latexmk　Visual Studio Code/LaTeX